

Unit -3

Chapter-4

THE ARCHITECTURE OF ARM 7

INTRODUCTION

- In the current world of embedded systems
- ARM processor has made its mark in almost all high-end devices
- It might be mobile phone, automotive hardware, factories or aircrafts
- The first popular one in this series is ARM7 which was used extensively in Apple iPods, PDAs (Personal Digital Assistant) and many other applications.

ARM AS A RISC PROCESSOR

- This powerful processor is designated as a RISC processor.
- RISC stands for Reduced Instruction Set Computer.
- A RISC architecture has only simple instructions realized in hardware.
- Complex operations are made possible by writing programs using these simple instructions.

- To elaborate, think of the 'divide' operation.
- RISC processors do not have an instruction for division
- A program that does repeated subtraction is a method of getting division done.

- The advantage is that the absence of a dedicated division unit reduces the complexity of the processor, and also leads to less power dissipation.
- The disadvantage is that division in RISC is not as fast as in CISC (Complex Instruction Set Computer).
- ARM design started as a pure RISC architecture.

Some of the features added to later generations, for making it a truly computationally intensive processor are

1. SIMD (Single Instruction Multiple Data) instructions
2. Floating Point Support
3. Specialized DSP (Digital Signal Processing) instructions

The main features of RISC are

This holds for most of the instructions

1. Instructions have the same size of 32 bits.
2. Each instruction takes one cycle to complete.
3. It is a 'load store' architecture.

The end result of the RISC approach made the processor to be a low power dissipating one and that became the reason why it easily conquered the growing embedded market, where hand held and battery operated devices are the major constituents.

ADDITIONAL FEATURES

As ARM was accepted in the embedded market, more features were added to it.

- ARM design was modular in structure so it was easy to add new features.
- The flexibility on account of the business model of ARM, made the addition of new and special features optional.
- Knowing these features will let us understand the early naming conventions of the different ARM chips.

1.THUMB SET

- The ARM Instruction Set Architecture has only 32-bit instructions
- This means that each and every instruction has to be of 32 bits.
- For simple applications, such a powerful set of instructions are not needed.

Thus, a new set of instructions, named THUMB, which are of 16 bits length was added.

- This is helpful in reducing the amount of code memory needed.

- The amount of the code in unit area of memory, that is, the code density, is more, when THUMB is used.
- Thumb code takes 40% less space in comparison to regular 32-bit ARM code but is slightly less efficient.
- There is also the facility to mix ARM and THUMB code and this is called ARM-THUMB interworking.

Cache

- This is a memory concept meant for speeding up memory accesses.
- Caches are always used in general purpose computing systems, but in the embedded domain, only high-end MCUs use it.
- The cache is a type of memory which is placed close to the processing engine, and is fast in response and low in volume.
- It is used to store a copy of important data and code which needs to be accessed at a rapid rate.
- When the required information is found in the cache is called a hit. Otherwise, it is a cache miss.
- It is not possible to guarantee that a required item will be found in cache when it is needed.
- Caches are not preferred for time critical applications where delays cannot be tolerated.
- In such systems ,a Tightly Coupled Memory(TCM) is considered to be a better option.

Cache

- In TCM's critical information(which is pre decided) is stored and used without the indeterminacy of caches.
- Cache storage is dynamic.
- It stores information that is likely to be frequently needed.
- Thus, the content of cache is continuously changed according to the decisions of what is considered important.
- TCMs contain static information that is predetermined to be very important .
- Critical procedures, interrupt routines etc are usually TCM hold.

2.CACHE

- All modern processors have caches, though some have only TCMs (Tightly Coupled Memories)
- ARM7 has 8 KB cache in which both instructions and data are allowed.

3.LONG MULTIPLIER

- Even though fast multiplication is a complex instruction, many ARM variants have such multipliers.

4.DEBUG UNIT

Inside the chip, there is a dedicated unit which provides the necessary support for testing and debugging.

- This unit is designed based on the specifications of JTAG (Joint Test Action Group).
- A 'boundary scan architecture' is defined which makes it easy to test the chip.

5. EMBEDDED ICE MACRO CELL

- All ARM processors need not have this feature.
- This is also a separate 'cell' to facilitate advanced debugging.

6.SYNTHESIZABLE

When the ARM IP is sold to the licensee in the form of an HDL (Hardware Definition Language)code, the buyer can flash this code to an FPGA (Field Programmable Gate Array), add peripherals, make changes etc.

Such ARM cores are said to be synthesizable'. Some FPGAs can also have this ARM design as a 'soft core' in it.

7.JAZELLE

In the early days of ARM, it was common for some ARM processors to execute Java bytecode in hardware as a third execution state along with the existing ARM and Thumb mode.

- This is useful to increase the execution speed of Java Micro Edition games and applications.
- Such an extension is now not in use.

8. ENHANCED DSP INSTRUCTIONS

- ARM to make useful in advanced signal processing applications, It was necessary to add advanced signal processing instructions in the processor.
- Such processors would have DSP instructions, which meant that specialized DSP hardware is available in them.

9. VECTOR FLOATING POINT UNIT

- Specialized hardware implies support for single and double precision arithmetic which enables floating point computations.

NAMING CONVENTIONS

- Naming convention is associated with the processor.
- The most popular ARM processor is ARM7 TDMI (Thumb Debug Multiplier In circuit Emulator)
- Which means an ARM7 processor with Thumb extension (T), Debug interface (D), long multiplier (M) and an ICE macro cell.(I). ARM7TDMI-E means that DSP extensions are available, and ARM7TDMI-S means that it is in synthesizable form.
- These naming conventions are not used for the new ARM processors, but many ARM7 processor capabilities can be gauged by this convention.

Table 4.1 Early naming conventions for ARM

ARM {x}{y}{z}{T}{D}{M}{I}{E}{J}{F}{H}{S}	
X	Family (7, 8, 9, 10, 11, ...)
Y	Memory management/protection unit
Z	Cache
T	Thumb 16-bit decoder
D	JTAG debug
M	Fast Multiplier
I	Embedded ICE macrocell
E	DSP Enhanced instructions (assumes TDMI)
J	Jazelle
F	Vector Floating-point Unit
S	Synthesizable Version

ARM Instruction Set Architecture Variants

- The first Instruction Set Architecture of ARM7 was called ARMv4.
- With a Thumb extension, it came to be called ARMv4T.

The next ISA was ARMv5 and used for ARM9 and it implied the Thumb set inherent in it.

- With DSP instructions added to it, the ISA became ARMv5E.

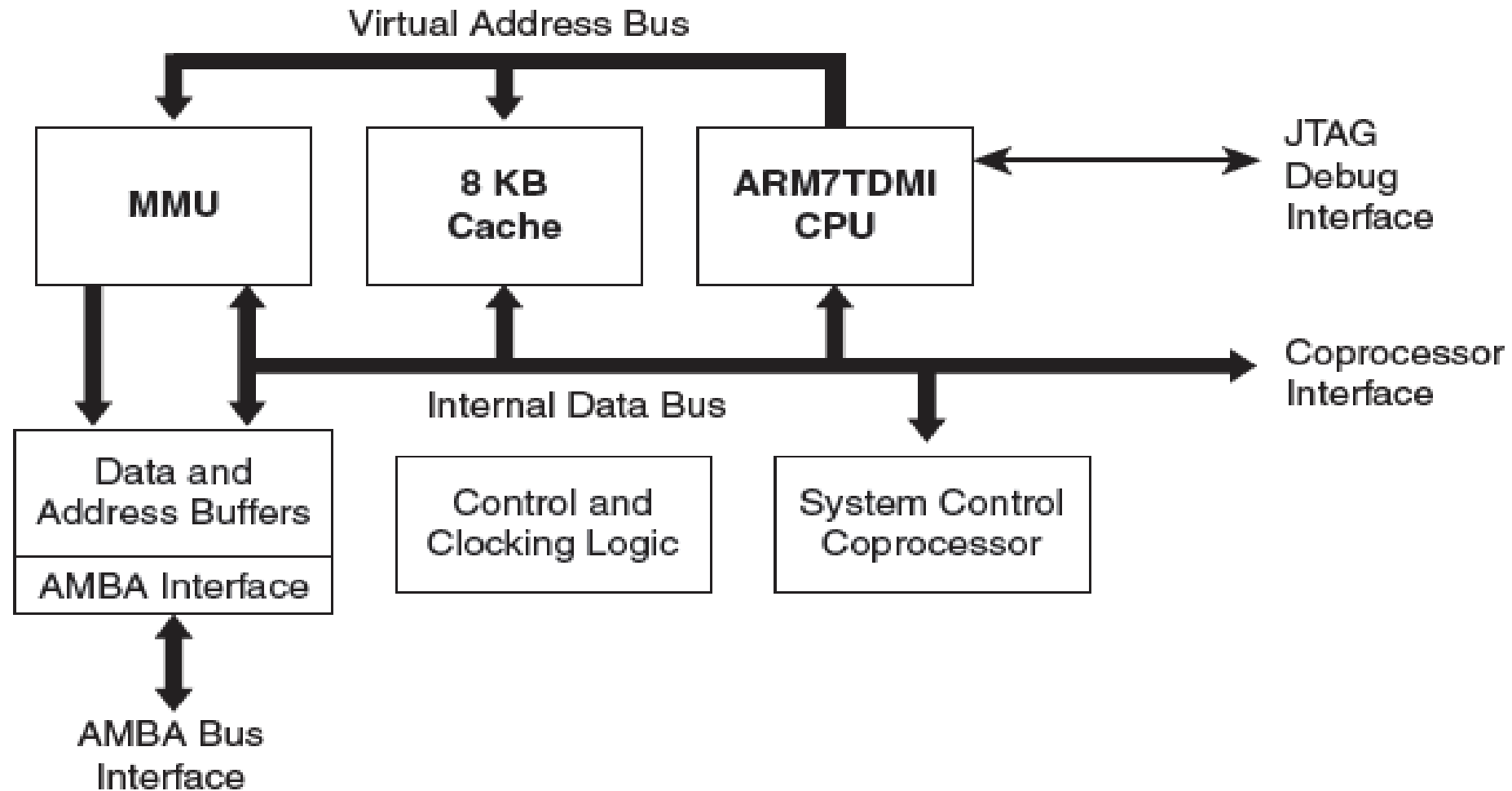
The next ISA series was ARMv6 and ARMv7 and used for ARM 11 as well as for the Cortex series.

- The latest ISA for 64 bit ARM is ARMv8.

4.2 ARM7 ARCHITECTURE

- We have seen the basic aspects of the ARM architecture
- Typical ARM7 processor as shown in the block diagram
- Note that it is an ARM7TDMI CPU.
- We start with the programmers model.
- A programmer model is the view of the processor in terms of its registers and its instruction set.
- It can operate in ARM and THUMB states.

BLOCK DIAGRAM OF A TYPICAL ARM7 PROCESSOR



- MMU- Memory Management Unit
- 8KB Cache -TCMs (Tightly Coupled Memories)

ARM7TDMI CPU- The most popular ARM processor is ARM7 TDMI (Thumb Debug Multiplier In circuit Emulator)

Data and Address Buffers

AMBA Interface- Advanced Microcontroller Bus Architecture

Control and Clocking Logic

System Control Co-Processor

PROGRAMMERS MODEL

A programmers model is the view of the processor, in terms of its registers and its instruction set.

- Since the CPU is ARM7 TDMI, we know that it can operate in the ARM and THUMB states.
- The ARM state is one in which it executes 32-bit, word-aligned ARM instructions.
The THUMB state is one in which it operates with 16-bit, half word-aligned THUMB Instructions.
- The data format in memory can be big endian or little endian
- This is selected by a bit named 'bigend' bit in the Control Register.
- It supports byte (8-bit), halfword (16-bit) and word (32-bit) data types.
- Words must be aligned to 4-byte boundaries and half words to 2-byte boundaries.

MODES OF OPERATIONS

- There are seven modes of operation as follows:

User: Unprivileged mode under which most tasks run.

FIQ (Fast Interrupt mode): Entered when a high priority (fast) interrupt is raised.

IRQ (Interrupt Request): Entered when a low priority (normal) interrupt is raised.

Supervisor: Entered on reset and when a Software Interrupt instruction is executed.

Abort: Entered when memory access violations occur.

Undef: Entered when an undefined instruction occurs.

System: Privileged mode of operation.

MODES OF OPERATIONS

Mode	Type	Example Use
User (USR)	Non-privileged	Running application code
FIQ	Interrupt	High-speed sensor handling
IRQ	Interrupt	Timer/keyboard interrupt
Supervisor (SVC)	Privileged	OS system call
Abort (ABT)	Exception	Invalid memory access
Undefined (UND)	Exception	Unknown instruction
System (SYS)	Privileged	OS kernel tasks

WHAT IS THE IMPORTANCE OF THESE MODES?

- All application programs run in the user mode.
The other modes, which are called 'exception modes' are entered for processing interrupts or for accessing protected resources.
- Mode changing may be done under 'software control' (by bit settings in registers) or by external interrupts or exceptions due to specific conditions or errors.
- Following Table indicates the specific exception through which each of the exception modes are entered.

Table 4.2 Exception modes

Mode on entry	Exception
Supervisor	Reset
Undefined	Undefined instruction
Supervisor	Software interrupt
Abort	Abort (prefetch)
Abort	Abort (data)
Reserved	Reserved
IRQ	IRQ
FIQ	FIQ

REGISTER SET

- ARM7 has a total of 37 registers
 - 31 general-purpose 32-bit registers
 - Six status registers
- All these registers cannot be seen at the same time.
- Some are available only in certain modes and states.

ARM REGISTER SET

ARM State General Registers and Program Counter

System & User	FIQ	Supervisor	Abort	IRQ	Undefined
R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7
R8	 R8_fiq	R8	R8	R8	R8
R9	 R9_fiq	R9	R9	R9	R9
R10	 R10_fiq	R10	R10	R10	R10
R11	 R11_fiq	R11	R11	R11	R11
R12	 R12_fiq	R12	R12	R12	R12
R13	 R13_fiq	 R13_svc	 R13_abt	 R13_irq	 R13_und
R14	 R14_fiq	 R14_svc	 R14_abt	 R14_irq	 R14_und
R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)

ARM State Program Status Registers

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
	 SPSR_fiq	 SPSR_svc	 SPSR_abt	 SPSR_irq	 SPSR_und

 = banked register

- The upper part of figure shows the general purpose registers and the Program counter.
- The same set of registers are used in the System and User modes.
- Certain registers are indicated as being 'banked'.
- The general registers in the FIQ mode R8 to R14 are notated as R8_fiq,..., R14_fiq.
- When a mode switch occurs from the user mode to the FIQ mode, the registers R8 to R14 of the user mode are replaced with a new set notated as R8_fiq, R9_fiq,..., R14_fiq.

- FIQ is a mode that is entered for a high priority 'fast interrupt request'.
- To enable fast response, no time is spent in 'saving the context' of the registers R8 to R14, there is no need to save the content of these registers and their content remains intact.
- The FIQ mode does not use these registers as it has an entirely new set of registers R8 to R14 to be used in this mode.
- In the other modes, only R13 and R14 are banked. Note that R13 and R14 are the 'Stack pointer register (SP)' and 'Link register (**LR**)' respectively of each mode.

The Uses of The Link Register:

- The link register is one that is not seen in most other processors.
- Its incorporation into ARM is meant to speed up branching during procedure calls and interrupt processing.
- Generally, in most processors, when a call or interrupt occurs, the 'return address' is saved in the stack which is in memory.
- This involves memory access which causes a certain amount of delay.
- In ARM, this delay is avoided by having this link register to save the return address.

Since all registers are inside the CPU, no memory access is necessary and thus an additional element of speed is obtained.

THE PROGRAM COUNTER REGISTER (PC)

- There is only one PC in any processor.
- The Program Counter is the register which sequences the instructions as they are being fetched and executed.
- In ARM, the value in the Program counter is the address of the current instruction being 'fetched' (rather than the address of the instruction being executed, as in other processors).

PROGRAM STATUS REGISTERS(PSR)

- The lower part of figure shows One CPSR (Current Program Status Register) and Banked SPSRs (Saved Program Status Register) for each mode.
- CPSR is the register which has the status bits and control bits for the 'current' time
- The bit configuration of the CPSR is shown in fig 4.3
- Bits 8-27 are 'reserved' which means they are undefined for ARM7.
- The upper five bits store the flag bits and are generally used to understand the result of a data processing instruction.

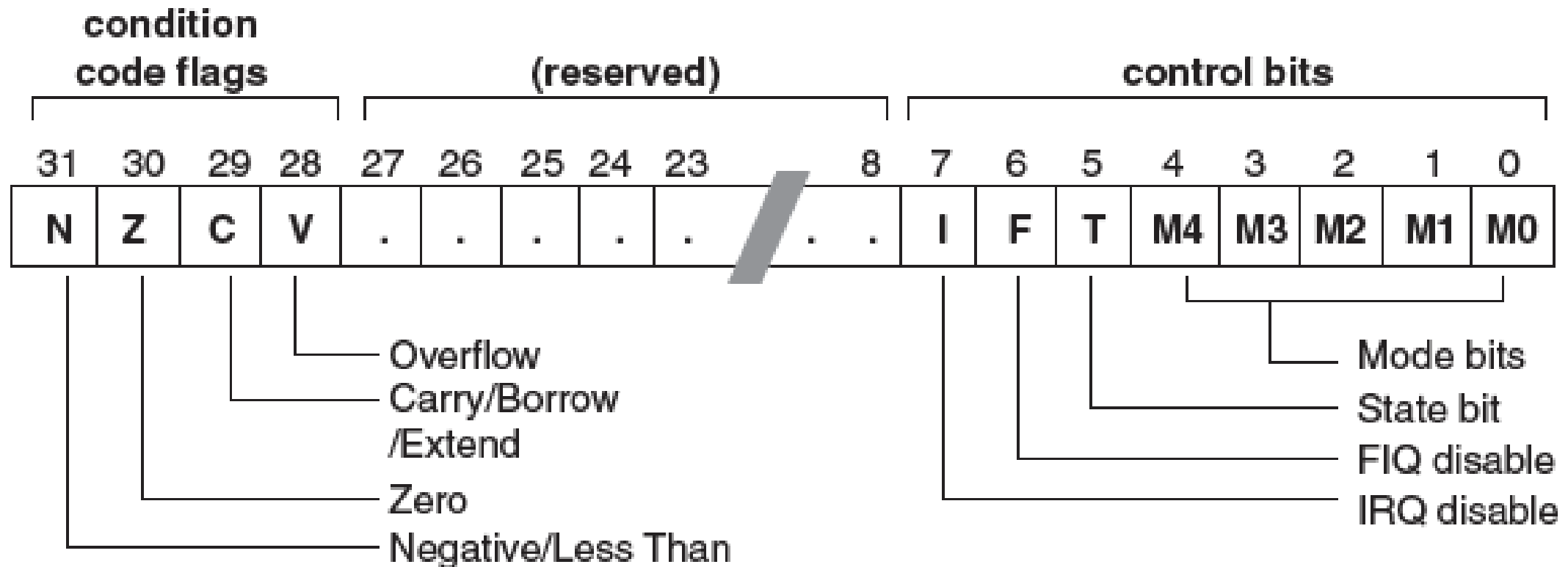


Figure 4.3 Bit configuration of CPSR (*Reproduced with permission from ARM Limited. Copyright © ARM Limited*)

- The lower eight bits of the CPSR are collectively known as the control bits.
- These change when an exception arises.
If the processor is operating in a privileged mode, they can also be manipulated by the software.
- The I and F bits, when set, are used for disabling the IRQ and FIQ interrupt modes.
- The T bit indicates the state of operation. If $T = 1$, the processor is in the Thumb state, otherwise, it is in the ARM state.

- The M0—M4 bits are used to indicate the mode of operation.
- Note that only seven modes are possible here, and any other number in the mode bits will cause an error from which an exit is possible only by a reset.

SAVED PROGRAM STATUS REGISTERS(SPSR)

There are five 'Saved Program Status Registers (SPSRs)', that is, one for each of the 'exception' modes of operation.

- These SPSRs are shown as banked registers
- When a mode switching occurs, the corresponding SPSR saves the current CPSR value into it.
- The system mode and user modes do not have SPSRs because they are not entered through exception mechanisms.

THE THUMB STATE REGISTERS

- The register set when the processor is in the Thumb state.
- This state involves 16-bit instructions
- It will still handle 32-bit data, but since low complexity applications are assumed for this state of operation, the number of registers used in this state is less.
- The THUMB state register set is a subset of the ARM state set.

The programmer has direct access to eight general purpose registers, R0—R7, the Program Counter (PC), a stack pointer register (SP), a link register (LR), and the CPSR.

- There are banked Stack Pointers, Link Registers and SPSRs for each exception mode.

THUMB State General Registers and Program Counter

System & User	FIQ	Supervisor	Abort	IRQ	Undefined
R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7
SP	 SP_fiq	 SP_svc	 SP_abt	 SP_irq	 SP_und
LR	 LR_fiq	 LR_svc	 LR_abt	 LR_irq	 LR_und
PC	PC	PC	PC	PC	PC

THUMB State Program Status Registers

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
	 SPSR_fiq	 SPSR_svc	 SPSR_abt	 SPSR_irq	 SPSR_und

 = *banked register*

Figure 4.4 The THUMB state registers (*Reproduced with permission from ARM Limited. Copyright © ARM Limited*)

THE RELATIONSHIP BETWEEN ARM AND THUMB STATE REGISTERS

- The THUMB state registers relate to the ARM state registers in the following way:
 1. THUMB state R0—R7 and ARM state R0—R7 are identical.
 2. THUMB state CPSR and SPSRs and ARM state CPSR and SPSRs are identical.
 3. THUMB state SP maps onto ARM state R13.
 4. THUMB state LR maps onto ARM state R14.
 5. The THUMB state Program Counter maps onto the ARM state Program Counter (R15).

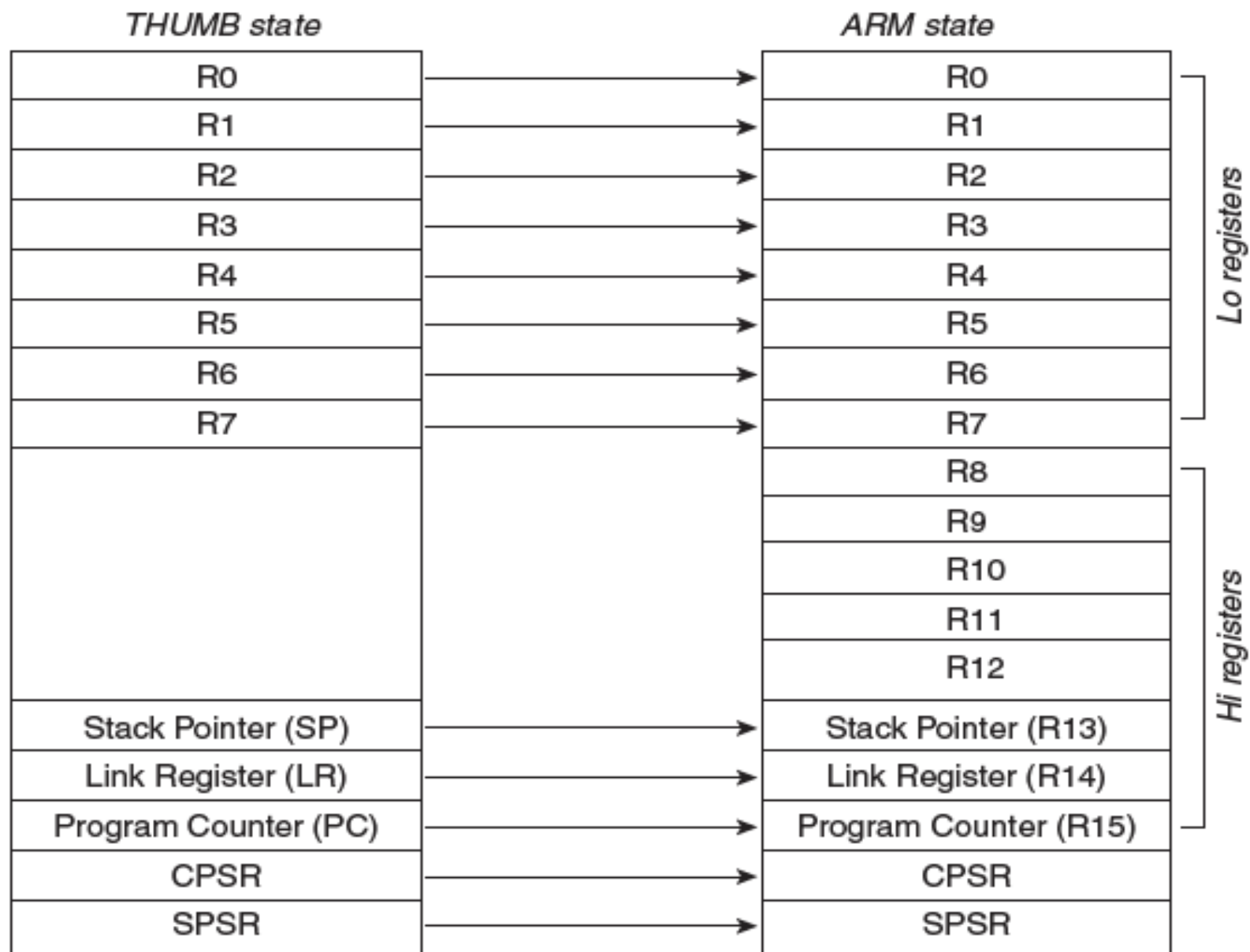


Figure 4.5 Relationship between the ARM and THUMB states (*Reproduced with permission from ARM Limited. Copyright © ARM Limited*)

4.3 INTERRUPTS AND EXCEPTIONS

- Exceptions/Interrupts are events that change the context of the processor
- They change the normal flow of the program
- The event may be triggered by software, a fault condition, a system call or an interrupt from a peripheral.
- The first thing to do when an exception occurs is to save the context of the processor
- Means that all working register contents must be saved
- The current content of the PC must also be saved

• THE PROCESSOR HANDLES AN EXCEPTION

1. The 'return address' which corresponds to the current PC value +8 is copied to the Link Register. The addition of 8 to PC is because the PC always contains the address of the instruction that is being fetched. Because of the three stage pipeline, the address of the instruction that is currently being executed is PC+8.
2. The content of CPSR is copied to the appropriate SPSR
3. The mode bits in the CPSR are changed to the mode of the exception mode to which switching has occurred.
4. The new PC value is the vector of the exception(table4.4)

Table 4.4 Exception vectors

Exception	Address
Reset	0x00000000
Undefined instruction	0x00000004
Software interrupt	0x00000008
Abort (prefetch)	0x0000000C
Abort (data)	0x00000010
Reserved	0x00000014
IRQ	0x00000018
FIO	0x0000001C

On Completion of the exception handler (ISR)

The following steps must be taken to return to the context of the interrupted program.

1. The link register contents are used to get back the PC value.
2. The SPSR is copied back to the CPSR.
3. The interrupt disable flags, if they were set on entry, are cleared.

EXCEPTION/INTERRUPT VECTORS

- Each exception (**interrupt**) has a specific address to which control branches to.
- This address is called the 'vector' of the exception.
- Interrupts which have predefined and fixed vectors, are generally designated as 'vectored interrupts'. Table shows the vectors of the exceptions defined for ARM7.

Table 4.4 Exception vectors

Exception	Address
Reset	0x00000000
Undefined instruction	0x00000004
Software interrupt	0x00000008
Abort (prefetch)	0x0000000C
Abort (data)	0x00000010
Reserved	0x00000014
IRQ	0x00000018
FIO	0x0000001C

EXCEPTION/INTERRUPT HANDLERS

What are the sources for interrupts?

- The current program running in the processor may be interrupted by an external device which requires service.
- A second case is when an **interrupt** occurs due to a software instruction.
- Another case is when an error occurs, like data abort or undefined instruction.
- In the first two cases, the new program **that comes** into play (called ISR or interrupt/ exception handler) performs the **service for the peripheral or acts upon** the interrupting instruction.

EXCEPTION IS GENERATED ON AN ERROR:

- Here also control branches to the vector of the exception.
The handler of the exception should contain the code which performs the necessary action to override the effect of the error.
- Thus, the system is prevented from crashing or going into a reset state before being able to save the current context.
- The ISRs of error-generated exceptions may be referred to as error handlers or fault handlers.
- These handlers allow the processor to exit gracefully from the error state without causing havoc to the system
- The different types of interrupts/Exceptions are

1.RESET

- When the processor is switched on it is in the Reset condition.
- All the hardware is initialized with a 'reset pulse'.
- The mode that is entered on reset is the 'Supervisor' mode.
- The first instruction that is to be executed then is taken from the address 0x00000000.

During the operation of the processors, reset may happen due to various reasons and then the system operation starts again from the reset vector.

2.UNDEFINED INSTRUCTION

- If an instruction is fetched and found to be a binary number which is not listed as one of the opcodes of the processor
- This exception is generated.
- The interrupt handler at the corresponding vector is to take appropriate action to exit gracefully from the error state.

3. SOFTWARE INTERRUPT

- The software interrupt instruction (SWI) [now changed to SVC- supervisor call] is used for entering Supervisor mode
- Usually to request a particular supervisor function.
- This is used by the OS to request access to protected resources.

4. ABORT

- An abort indicates that the current memory access cannot be completed.
- There are two types of abort:
 1. Prefetch abort occurs during an instruction prefetch.
 2. Data abort occurs during a data access (R/W), when data at an invalid address is attempted to be accessed.

5. PREFETCH ABORT

- It occurs when the processor prefetches code (could be a case of branching) which is at an invalid address.
- It may be that this address is in a protected memory area,
- This address is a peripheral address which has not been mapped to a peripheral.

If a prefetch abort occurs, the prefetched instruction is marked as invalid, but the exception will not happen until the instruction reaches the execution unit of the pipeline.

6. RESERVED

Table 4.4 shows a vector for an exception that is not used for ARM7 but may be used for later processors.

7. IRQ

- The Interrupt Request (IRQ) exception is a normal interrupt caused by a LOW level on the IRQ pin.
- It is generally used by peripherals to get service.

8. FAST INTERRUPT REQUEST

- This is used for very important actions and is designed to be very fast.
- When this interrupt occurs, the processor goes to the FIQ mode, where a new set of registers are used, so as to avoid the delay for context saving.

In practical situations, the applications which use the FIQ exception are coded in assembly to bring in an additional level of speed.

4.3.3 PRIORITY OF EXCEPTIONS

- When multiple exceptions arise at the same time, there is a priority prefixed to determine the order in which they are handled.
- The order is as follows:
 1. Reset (Highest priority)
 2. Data abort
 3. FIQ
 4. IRQ
 5. Prefetch abort
 6. Undefined Instruction, Software interrupt (Lowest priority)

All Exceptions cannot occur at the same time

- Undefined instructions and software interrupts are mutually exclusive.
- If an instruction is decoded and found to be an SWI (software interrupt instruction), it cannot be an undefined instruction.
- A data abort exception occurs at the same time as an FIQ.
- The processor first enters the data abort handler and then immediately after that proceeds to the FIQ vector.
- A normal return from FIQ will cause the data abort handler to resume execution.
- It is necessary to place the data abort exception at a higher priority than FIQ to ensure that the data transfer error does not escape detection.

4.4 ARM7 PIPELINE

- The idea of pipelining is well known in computer architecture.
- The simplest is a three-stage pipeline to overlap the basic operations of fetch, decode and execute.
- ARM 1 used a three-stage pipeline and this has been continued for ARM7.

ARM9 uses a five-stage pipeline and ARM 10 uses a six-stage pipeline.

FIGURES SHOW THE THREE-STAGE PIPELINE OF ARM7.



Figure 4.6 Three-stage pipeline of ARM 7

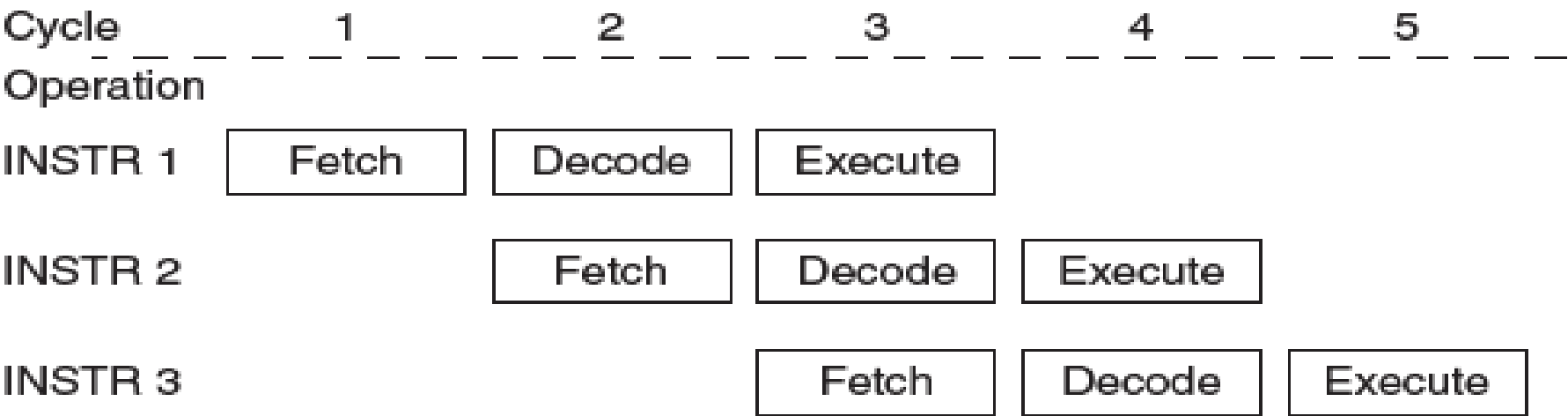


Figure 4.7 Pipeline filling in ARM 7

ARM9 and ARM10

- ❑ ARM9 processor is a 5 stage (five clock cycle) ARM state pipeline is used, consisting of FETCH, DECODE, EXECUTE, MEMORY and WRITE BACK stages.
- ❑ ARM10 processor is a 6 stage consisting of FETCH ,ISSUE, DECODE,EXECUTE, MEMORY ,WRITE stages.

4.5 ADVANCED FEATURES

4.5.1 AMBA

- AMBA is associated with ARM and it is a widely used interconnection standard for System on Chip (SoC) design.
- AMBA stands for Advanced Microcontroller Bus Architecture.
- In 1996 it was developed as a standard bus to facilitate the concept of modularity when adding on-chip functional blocks like memory and peripherals.
- The AMBA specification has become a well-accepted de-facto standard for the semiconductor industry
- used by 95% of ARM's partners and a number of IP providers.

AMBA Contd...

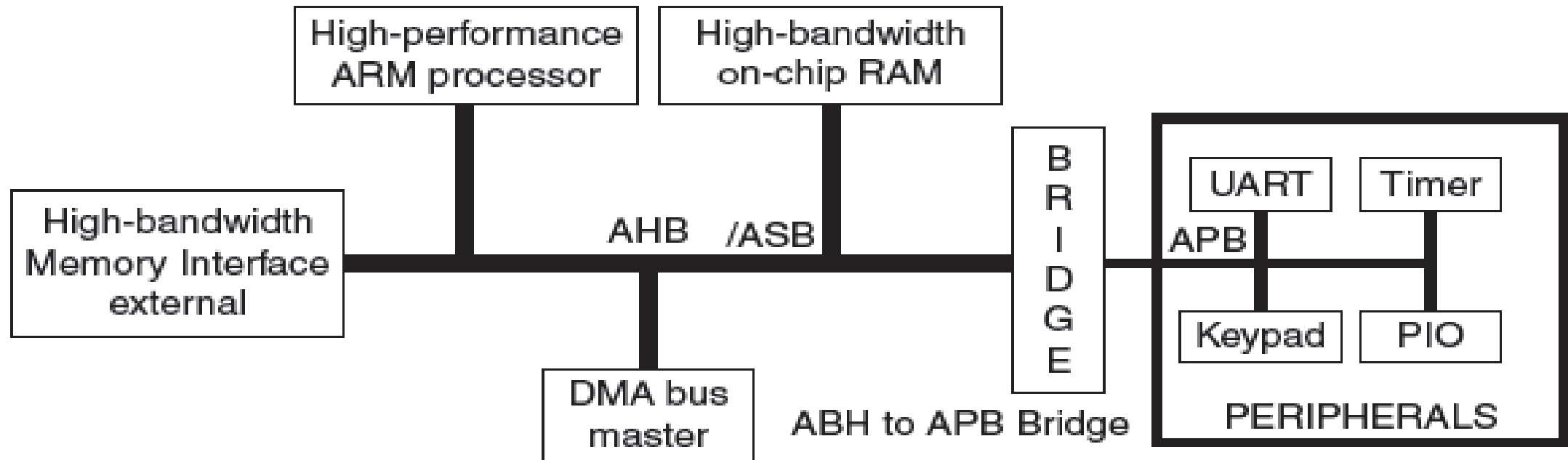
The AMBA interface is a processor and technology independent, and so the reusability of peripheral and system components is ensured across a wide range of applications.

The AMBA specification defines three buses

1. Advanced System Bus (ASB)
2. Advanced High-performance Bus (AHB)
3. Advanced Peripheral Bus (APB).

Figure shows that the high-performance modules are on the AHB/ ASB bus. There is a bridge which converts this standard to a lower speed peripheral bus.

COMPONENTS OF AMBA BUS



AMBA Advanced High-performance Bus (AHB)

* High performance

AMBA Advanced Peripheral Bus (APB)

* Low power

Figure 4.8 Components of AMBA bus

These Buses are used in

- The AHB is the backbone of the system and is designed specifically for high performance, high-frequency components. This includes the connections of processors, on-chip memories, and memory interfaces among others.
- AHB is able to sustain the external memory bandwidth, on which the CPU, on-chip memory and other Direct Memory Access (DMA) devices reside. This part has to be a high-speed bus.
- The ASB is an alternative to the AHB where some high-performance features are not needed.

Contd...

- The APB is a simplified interface designed for low bandwidth peripherals that do not require the high performance of the AHB or the ASB.
- APB is the part which is used by the on-chip peripherals and operates at a lower speed.
- These include components like a UART, low-frequency GPIO, and timers.

- In 2003, ARM introduced the 3rd generation, AMBA 3, AMBA revision 3.0 introduced the AXI(Advanced eXtensible Interface) protocol to reach even higher performance interconnect and the Advanced Trace Bus (ATB) as part of the Core Sight on-chip debug and trace solution.
- This is used in Cortex processors, not on ARM7.

4.5.2 COPROCESSORS

- For ARM7 to ARM 11, additional and sometimes optional functions were obtained by the concept of coprocessors
- A coprocessor is an additional functional block which has its own instruction set

When some function needs to be done, which is not supported by the general ARM core, a coprocessor is brought in to do it

- For example, the ARM core is not capable of high-end floating point arithmetic processing.

- When the need of such processing is needed, an arithmetic coprocessor is used, which operates together at the same time with the main core.
- Similar to this, there are coprocessors for the control of MMU, cache, DSP etc.
- A licensee of ARM, when designing a chip, has the option to add the coprocessor modules that is needed for his design.

- Each coprocessor has its own functional hardware with its instructions.
- Up to 16 coprocessors (CPO to C15) have been defined in the ARM cores.
- Some of the important architectures. All of them are not present in all core coprocessors of ARM7 are as follows:
 1. CP10: Vector Floating Point unit
 2. CP11: SIMD hardware and software named NEON
 3. CP14: Debug unit
 4. CP15: System control coprocessor for memory and cache management (including TCM)

The Cortex series does not use the concept of coprocessor.

4.5.3 MEMORY MANAGEMENT AND MEMORY PROTECTION

- Many ARM cores have only a memory protection unit (MPU).
- More advanced ARM chips have a memory management unit which includes protection features as well.
- Here the MMU of a typical ARM7 chip. ARM710T is one such processor.
- The MMU performs two primary functions:
 1. Translation of virtual addresses to physical addresses
 2. Provision of protection to memory by stipulating 'access permissions'

- The MMU has the following dedicated hardware to perform these functions:
 1. Translation Lookaside memory (TLB)
 2. Access control Logic
 3. Translation table walking logic
- ARM7 memory sizes are designated as either sections or pages.
- Sections are 1 MB blocks of memory. Pages may be small of size 4 KB, or large of size 64KB

Within the small page, there can be tiny pages (sub-pages) of size 1KB.

Within a large page, there can be 16KB sub-pages.

The sub Pages are meant to control access permission within a memory area which is smaller than the specified page.

The MMU also supports the concept of domains — which are areas of memory that can be defined to possess individual access rights.

The domain Access Control Register is used to specify access rights for up to 16 separate domains.

4.5.3.1 USING THE Translation Lookaside Buffer

- A Translation Lookaside Buffer stores a certain number of translations of logical addresses to physical addresses.
- For the ARM7 MMU, 64 translated entries are saved in the TLB.
- If the required translation is available in the TLB, the 'access control logic' performs a permission check to verify if access is to be allowed.

If access is permitted, the MMU outputs the appropriate physical address corresponding to the virtual address.

If access is not permitted, the MMU signals the CPU to go to the abort mode through the abort exception.

4.5.3.1 USING THE TLB

- If the TLB does not contain the physical address corresponding to the generated virtual address, the 'Address Translation Logic' which is referred to as 'translation table walk hardware' takes over.
- This action calculates the physical address corresponding to the given logical(virtual)address.
- Memory calculation is done after the physical address is calculated.
- The results of address calculation is also placed in the TLB, overwriting an existing value there in.

4.5.3.2 Address Translation Process

4.5.3.2 *Address translation process*

The MMU translates the virtual addresses generated by the CPU into physical addresses to access memory, and also ensures that checks on the access permission is done before access is begun.

The address translation data and the access permission data is available in a table and is in physical memory. The MMU is the hardware which uses this information for the following purposes.

- To walk through this translation table
- Obtain the translated physical address
- Confirm that the access permission conditions are met

There are three routes by which the address translation (along with permission check) takes place. The translation route starts in the same way, irrespective of whether a section or a page is to be accessed. The difference is that a section translation is a one-step process, while page access needs two steps, that is, there are two levels of translation for page access.